Scan Finder — How Detection Works, & Today's Work

A plain-English map of the pipeline · 24 July 2026

> Scan Finder turns a scanned document into filed data in a **staged pipeline**. Each stage adds or checks one thing, and passes the document to the next. The golden rule underneath all of it: **when unsure, send it to review — never file a silent wrong value.** This page shows what each stage does, the decision it makes, and what changed today. Items marked ★ are today's fixes.

1 What we did today

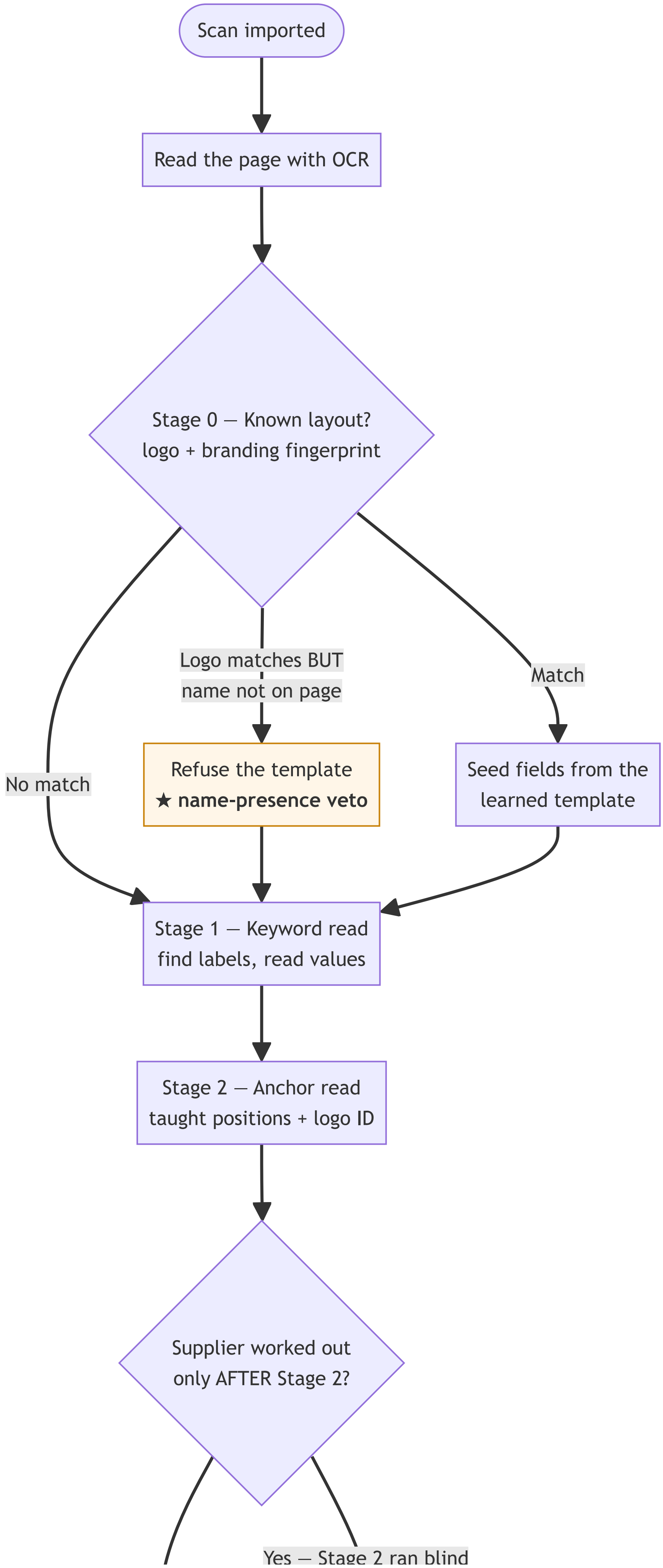
Four fixes shipped (three live, one held safely behind a gate) plus two "measured → do nothing" calls and one earlier straighten fix. Every risky change has an on/off switch.

Fix	What it does, in plain terms	Status
Own-label exemption (issue 2)	A value read from a precise label ("Invoice No", "PO Date") is no longer wrongly flagged just because a taught position didn't line up. Shared/generic labels ("Date", "#") are still checked.	LIVE
Late located-recovery	When the supplier's name is worked out *late*, the taught anchor that never got to run is re-run to find the label and confirm the value — so a correct ref/date stops being held for no reason.	LIVE
Name-presence veto	Kills a wrong "Template available: *Other Supplier*" suggestion. If a supplier reliably prints its own name but that name isn't on this page, its template can't be suggested (fixed Larkspur showing on a Saltmarsh doc).	LIVE
Name-guard clear	Clears a false "caption disagreed" flag on a correct name when an independent read agrees. **Built and correct, but switched OFF** — it can expose a separate ref misread until the ref-guard below lands.	DARK
Straighten witness	Straighten + reprocess no longer silently corrupts a value (a rotated crop that flipped a digit is caught by re-reading the original scan).	LIVE
Two "do-nothing" calls	Measured a cross-tier auto-file lift and the "Slice 1d" logo veto — both turned out to add nothing safe or already ship. Reverted rather than add risk.	MEASURED

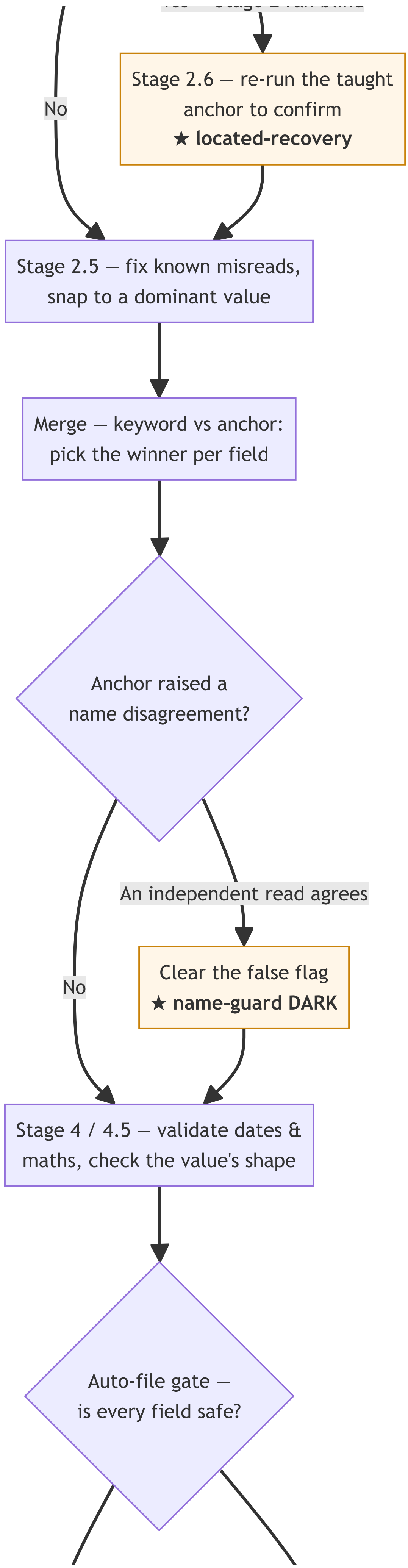
Next up (the one tracked follow-up): a **ref-guard** that flags a valid-shaped single-digit ref disagreement between the tight crop and the full page (e.g. `DN-28472` vs `DN-38472`). It's the missing safety net that lets the name-guard clear switch on — and it fixes silent ref misreads on its own.

2 The pipeline at a glance

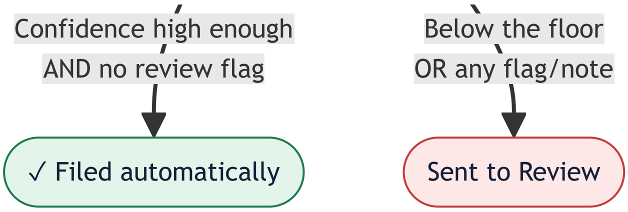
Each box is a stage; each diamond is a decision. Follow the arrows top to bottom. ★ marks where today's fixes live.



Scan imported
Read the page with OCR
Stage 0 — Known layout?
logo + branding fingerprint
No match
Logo matches BUT
name not on page
Match
Refuse the template
★ name-presence veto
Seed fields from the
learned template
Stage 1 — Keyword read
find labels, read values
Stage 2 — Anchor read
taught positions + logo ID
Supplier worked out
only AFTER Stage 2?
Yes — Stage 2 ran blind



No
Stage 2.6 — re-run the taught anchor to confirm
★ **located-recovery**
Stage 2.5 — fix known misreads, snap to a dominant value
Merge — keyword vs anchor: pick the winner per field
Anchor raised a name disagreement?
An independent read agrees
No
Clear the false flag
★ **name-guard DARK**
Stage 4 / 4.5 — validate dates & maths, check the value's shape
Auto-file gate — is every field safe?



3 Each stage, in plain English

What it does · **the decision it makes** · where detection stands.

OCR — read the page `tesseract`

Turns the scan into text with positions. If the page is crooked it can be "straightened" first.

Decision: none yet — it just produces the raw words and where they sit.

Today: straightening can occasionally corrupt a tight crop; that's now caught by re-reading the original scan (the "straighten witness").

Stage 0 — Template match `template_matcher.py`

Asks "have I seen this supplier's exact layout before?" using the logo shape and the distinctive words on the page. If yes, it pre-fills the fields from the learned template.

Decision: use a learned template, suggest one, or start fresh.

Where it stands: logo shapes alone **cannot** reliably tell suppliers apart on scans (two different logos can look identical to the maths). So today's **name-presence veto** makes the page *text* the tie-breaker — a supplier that prints its name can't be suggested for a page missing it.

Stage 0.5 — Taught mappings `template_mapper.py`

If you've drawn a box around a field ("teach"), this reads that exact spot, adjusting for small shifts.

Decision: read the taught box; fall back to relocation if the layout moved.

Stage 1 — Keyword read `keyword.py`

The workhorse. Finds labels ("Invoice No", "Order Date", "Customer") and reads the value next to them. Handles ~60–70% of fields on its own.

Decision: for each field, which label matched and what value sits by it.

Today: the **own-label exemption** means a value read from a field's *own* distinctive label is trusted, not wrongly held.

Stage 2 — Anchor read + logo ID `anchor.py`

Uses positions you've taught (per supplier) and the logo to identify the sender. Recovers when things drift, and cross-checks a tight crop against the full page.

Decision: the taught value, the supplier's identity, and whether two reads of a field *agree*.

Where it stands: this is the trickiest stage. A "relocated" crop can land on junk or on a caption — today's work makes disagreements route to review honestly, and (dark) clears them when an independent read confirms the value.

Stage 2.6 — Late rescue `engine.py`

If the supplier's name was only worked out *after* Stage 2 already ran, the taught anchors never got a chance. This re-runs just those, finds the label, and confirms the value.

Decision: does a genuine "located" read confirm the value? If yes, stop holding it.

Today: this is the **located-recovery** fix — it lifts a correct ref/date that was being held only because the anchor ran too early.

Stage 2.5 — Corrections `ocr_corrector.py`

Applies learned fixes (a repeated misread) and can "snap" a value to the dominant confirmed value when the same supplier always uses it.

Decision: replace a known misread with the value history proves is right.

Stage 4 & 4.5 — Validate + format check `validator.py / format_anomaly_checker.py`

Normalises dates, checks totals add up, and compares each value's *shape* to the supplier's history (a ref that suddenly changes pattern is suspicious).

Decision: is each value well-formed and consistent with what this supplier normally sends?

The auto-file gate `trust.js`

The final safety check. A document files itself only if **every** valued field is confident enough and carries **no** review flag. New suppliers must earn trust (a few clean confirms) before anything auto-files.

Decision: auto-file, or send to Review — the last line before your eyes.

Any single "please verify" note holds the whole document. That's why today's care was to remove only *false* flags, never a real one.

4 The golden rule

> **Fail toward review, never toward a silent wrong value.** One document filed with the wrong number is worse than ten honest "please check this" holds. Every change today was measured against that: three fixes removed *needless* reviews without ever letting a wrong value slip through, and the one fix that *could* have (the name-guard clear) was switched off until its safety net is built.

Scan Finder · detection overview generated 24 Jul 2026 · branch `feat/reprocess-throughput-autostraighten` (pushed). Kill switches for every fix: `TAUGHT_OWNERSHIP_OWN_LABEL`, `LATE_RESCUE_LOCATED_CORROB`, `TEMPLATE_NAME_PRESENCE_VETO`, `NAME_GUARD_KEYWORD_CLEAR` (off), `DESKEW_RAW_WITNESS`.